Day 2_Machine Learning

Day 2

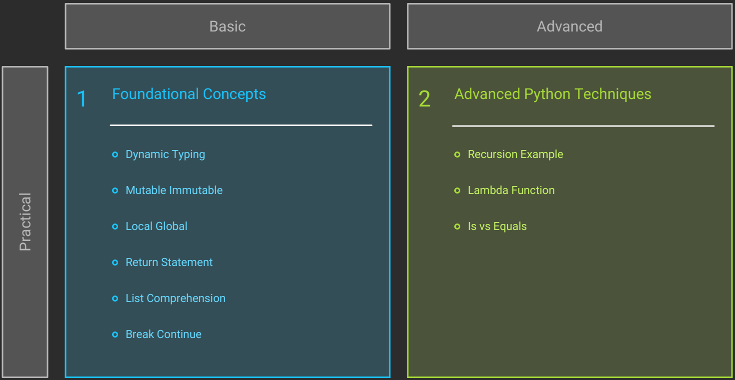
PART 1: Python (10 Questions)

1. What is dynamic typing in Python?
2. What is the difference between mutable and immutable data types?
3. What are local and global variables?
4. What is the purpose of the return statement in a function?
5. What is list comprehension?
6. Difference between break, continue, and pass
7. What is recursion? Give a simple example.
8. What is a lambda function?
9. What is the difference between is and ==?
10. What are docstrings and why are they important?

PART 2: NumPy

1. What is vectorization in NumPy?
2. What is the difference between reshape() and resize()?
3. What are NumPy dimensions and axes?
4. What is slicing in NumPy arrays?
5. How does NumPy help in mathematical computations for AI?

Python & NumPy Skills



Basic

Advanced

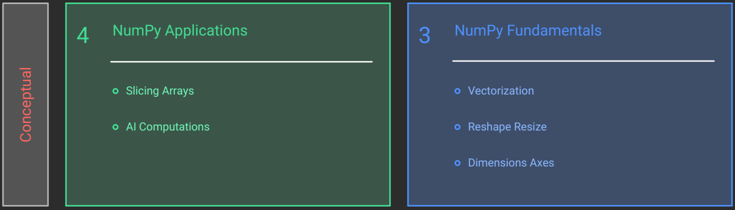
Practical

1 Foundational Concepts

- Dynamic Typing
- Mutable Immutable
- Local Global
- Return Statement
- List Comprehension
- Break Continue

2 Advanced Python Techniques

- Recursion Example
- Lambda Function
- Is vs Equals



Conceptual

4 NumPy Applications

- Slicing Arrays
- AI Computations

3 NumPy Fundamentals

- Vectorization
- Reshape Resize
- Dimensions Axes

Made with Napkin

Python Coding Tasks

1. Dynamic Typing

Write a program that:

* Assigns an integer value to a variable
* Reassigns a string value to the same variable
* Prints the value and its type at each step

```
x = 10
print(x, type(x))


x = "Hello Python"
print(x, type(x))
```

2. Mutable vs Immutable Data Types

Write a program that:

* Modifies a list
* Attempts to modify a tuple

```
my_list = [1, 2, 3]
my_list[0] = 10
print(my_list)


my_tuple = (1, 2, 3)
# my_tuple[0] = 10
print(my_tuple)
```

3. Local and Global Variables

Write a program that:

* Uses a global variable
* Uses a local variable inside a function

```
x = 100


def demo():
    y = 50
    print(x, y)


demo()
```

4. Return Statement in a Function

Write a function that:

* Takes two numbers

* Returns their sum

```
def add(a, b):
    return a + b


result = add(5, 3)
print(result)
```

5. List Comprehension

Write a program that:

* Creates a list of squares from 1 to 10

```
squares = [x * x for x in range(1, 11)]
print(squares)
```

6. break, continue, and pass

Write a program that:

* Demonstrates break, continue, and pass

```
for i in range(1, 11):
    if i == 3:
        continue
    if i == 5:
        break
    if i == 7:
        pass
    print(i)
```

7. Recursion

Write a recursive function that:

* Finds the factorial of a number

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)


print(factorial(5))
```

8. Lambda Function

Write a lambda function that:

* Returns the square of a number

```
square = lambda x: x * x
print(square(4))
```

9. is vs ==

Write a program that:

* Compares two lists using == and is

```
a = [1, 2, 3]
b = [1, 2, 3]


print(a == b)
print(a is b)
```

10. Docstrings

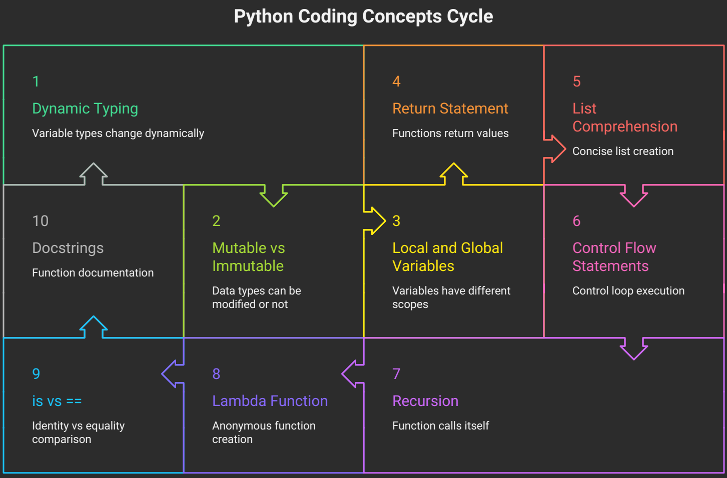
Write a function that:

* Uses a docstring

```
def greet():
    """This function prints a greeting message"""
```

```
    print["Hello Python"]


greet[]
print[greet._doc_]
```



Python Coding Concepts Cycle

1
Dynamic Typing
Variable types change dynamically

2
Mutable vs Immutable
Data types can be modified or not

3
Local and Global Variables
Variables have different scopes

4
Return Statement
Functions return values

5
List Comprehension
Concise list creation

6
Control Flow Statements
Control loop execution

7
Recursion
Function calls itself

8
Lambda Function
Anonymous function creation

9
is vs ==
Identity vs equality comparison

10
Docstrings
Function documentation

Made with Napkin

PART 3: Pandas

1. What is data cleaning in Pandas?
2. What is the difference between loc and iloc?
3. What are missing values and how does Pandas handle them?
4. What is groupby[] and why is it used?
5. How does Pandas help in data preprocessing for AI models?

Pandas Practice

Goal: Learn Pandas step by step using real datasets created with NumPy
Common Setup [Run Once]

```
import numpy as np
import pandas as pd
```

DATASET 1: Titanic-Like Passenger Dataset

Purpose: Learn data inspection, filtering, missing values, grouping

Create Dataset using NumPy

```
np.random.seed(1)
rows = 200


df = pd.DataFrame({
    "PassengerId": np.arange(1, rows + 1),
    "Name": np.random.choice(["Mr. John", "Mrs. Anna", "Miss Emma", "Dr. Smith"], rows),
    "Age": np.random.choice(np.append(np.random.randint(1, 80, 180), [np.nan]*20), rows),
    "Sex": np.random.choice(["male", "female"], rows),
    "Pclass": np.random.choice([1, 2, 3], rows),
    "Fare": np.round(np.random.uniform(10, 500, rows), 2)
})
```

View Data

```
df.head()       # View first 5 rows
df.tail()       # View last 5 rows
df.sample(3)    # View random 3 rows
```

Inspect Structure

```
df.shape        # Number of rows and columns
df.columns      # Column names
df.info()       # Data types and missing values
df.describe()   # Statistical summary
```

Handle Missing Data

```
df.isnull().sum()               # Count missing values
df["Age"].fillna(df["Age"].mean(), inplace=True)
```

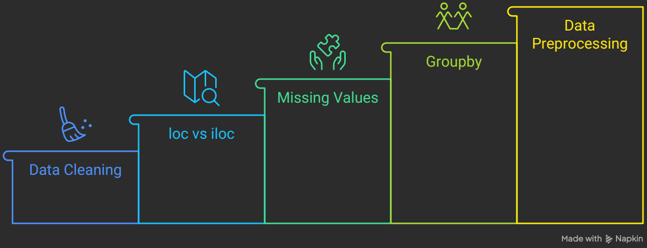
Filtering Rows

```
df[df["Age"] > 30]
df[df["Sex"] == "female"]
```

Grouping & Aggregation

```
df.groupby("Pclass")["Fare"].mean()
```

Mastering Pandas for Data Preprocessing



DATASET 2: Student Performance Dataset

Purpose: Sorting, value counts, apply, conditional columns

Create Dataset

```
np.random.seed(2)
rows = 150
```

```
df = pd.DataFrame({
    "StudentId": np.arange(1, rows + 1),
    "Age": np.random.randint(18, 30, rows),
    "Gender": np.random.choice(["male", "female"], rows),
    "Course": np.random.choice(["AI", "DS", "ML", "Python"], rows),
    "Marks": np.random.randint(35, 100, rows)
})
```

Sorting Data

```
df.sort_values(by="Marks", ascending=False)
```

Value Counts

```
df["Gender"].value_counts()
```

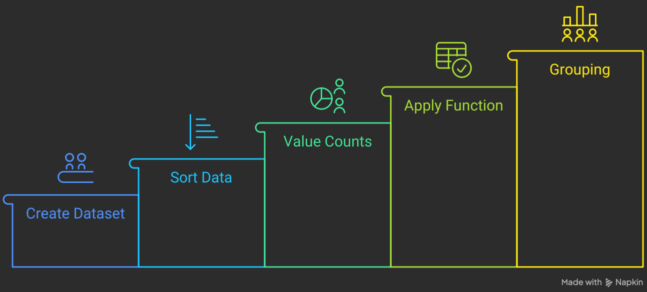
Apply Function

```
df["Result"] = df["Marks"].apply(lambda x: "Pass" if x >= 50 else "Fail")
```

Grouping

```
df.groupby("Course")["Marks"].mean()
```

Analyzing Student Performance



DATASET 3: Employee Dataset

Purpose: Rename, drop, mapping, aggregation

Create Dataset

```
np.random.seed(3)
rows = 120
```

```
df = pd.DataFrame({
    "EmpId": np.arange(1, rows + 1),
    "Dept": np.random.choice(["HR", "IT", "Sales"], rows),
    "Salary": np.random.randint(30000, 150000, rows),
    "Gender": np.random.choice(["male", "female"], rows)
})
```

Rename & Drop

```
df.rename(columns={"Dept": "Department"}, inplace=True)
df.drop("Gender", axis=1, inplace=True)
```

Aggregation

```
df.groupby("Department")["Salary"].agg(["mean", "max"])
```